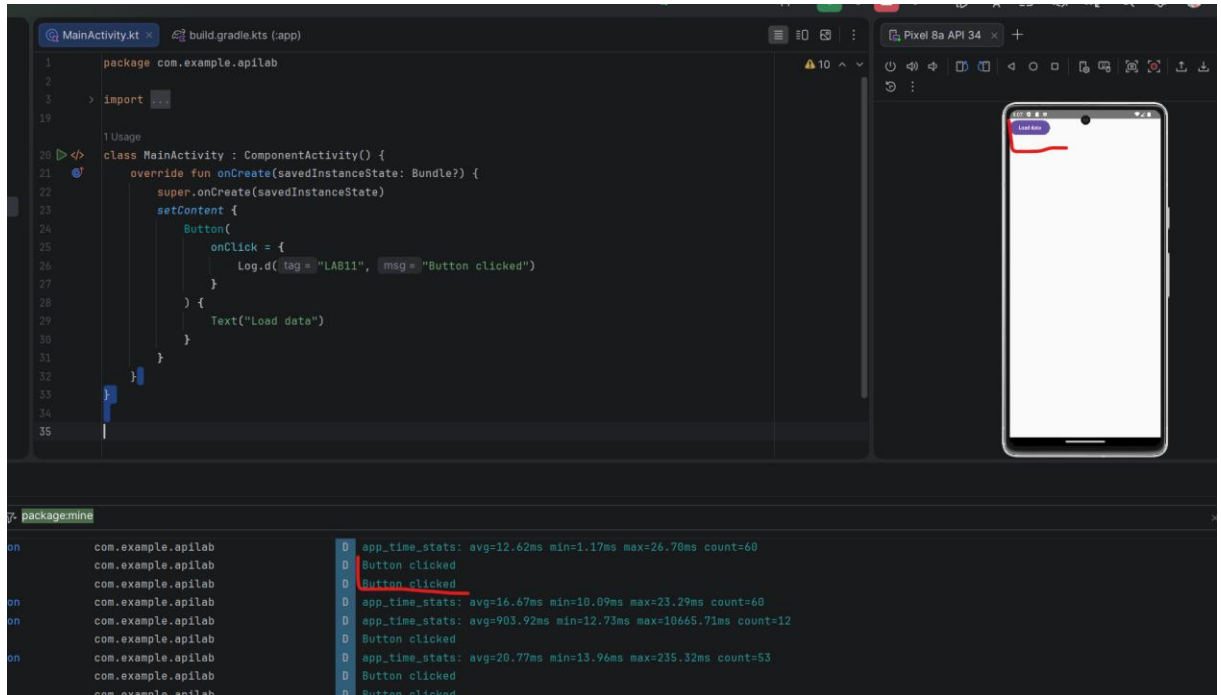
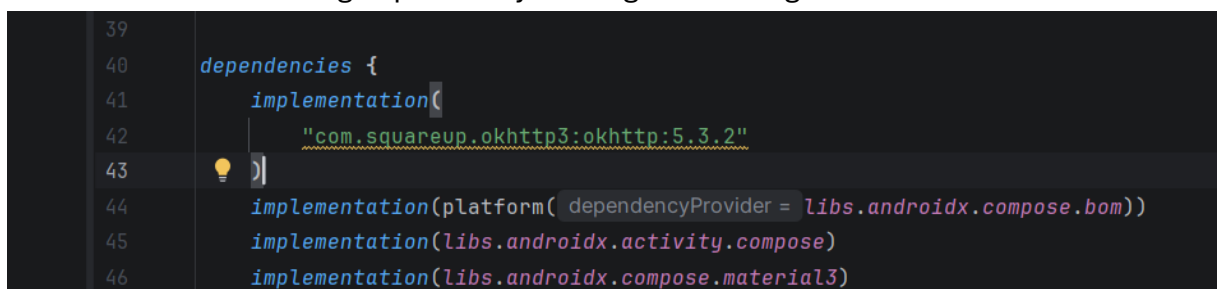


Mobile App Development – working with external APIs

- Let's create the empty activity project called API-practice with API 34 as the minimum sdk
- Let's clean up the pre-generated Main Activity class just by setting up the following set content and let's launch the app just to verify that everything is correct



- The typical schema of working with the external APIs is the following
 - Android App → HTTP Request → JSON → Kotlin Objects → UI
- To make/ create the HTTP Request we will use the squareup.okhttp3 library so we need to add the following dependency to our gradle configuration file



- For practicing we will build a “post explorer” using the well-known service/api: <https://jsonplaceholder.typicode.com/>
- Let's then create our first request to download the single post from the external API by adding in Main Activity the following function:

```

fun downloadPost(){
    Thread{
        try{
            val client= OkHttpClient()
            val request= Request.Builder()
                .url(
                    "https://jsonplaceholder.typicode.com/posts/1"
                )
                .build()
            val response=
                client.newCall(request)
                    .execute()
            val result=
                response.body?.string()
            Log.d(
                tag = "LAB11",
                msg = result.toString()
            )
        }catch(e:Exception){
            Log.d(
                tag = "LAB11",
                msg = e.message.toString()
            )
        }
    }.start()
}

```

- Add calling this function to our onClick event:

```

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView {
            Button(
                onClick={
                    Log.d( tag = "LAB11", msg = "Button clicked")
                    downloadPost()
                }
            ){
                Text("Load data")
            }
        }
    }
}

```

- And let's verify that it works and you should get something like:

```

ProfileInstaller  com.example.apilab  Installing profile for com.example.apilab
LAB11            com.example.apilab  {
                                "userId": 1,
                                "id": 1,
                                "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
                                "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum
                                }

```

- Let's modify our downloadPost function by adding the following fragment:

```

        response.body?.string()
        Log.d(
            tag = "LAB11",
            msg = result.toString()
        )

        val json = JSONObject(
            result.toString()
        )
        val title =
            json.getString( name = "title")
        Log.d(
            tag = "LAB11",
            msg = title
        )
    } catch (e: Exception) {

```

- And let's what we are getting right now:

```

2020-06-03 11:26:57.738 18371-18407 LAB11 com.example.apilab {
  "id": 1,
  "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
  "body": "quia et suscipit? suscipit recusandae consequuntur expedito et culpa reprehenderit? molestiae ut ut quas tota? nulla in rerum est autem sunt rem tenetur nostrum nostrum."
}
2020-06-03 11:26:57.738 18371-18407 LAB11 com.example.apilab {
  "id": 1,
  "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
  "body": "quia et suscipit? suscipit recusandae consequuntur expedito et culpa reprehenderit? molestiae ut ut quas tota? nulla in rerum est autem sunt rem tenetur nostrum nostrum."
}

```

- So now, along with the “raw” api answer you should see just the title of the post
- Let's now convert the response to the objects. So first we need to create the data class, let's say Post, e.g. with id, tile and content like this:

```

1 package com.example.apilab
2
3 data class Post(
4     val id:Int,
5     val title:String,
6     val body:String
7 )
8

```

- Now we may use that in our downloadPost function:

```

        val post=Post(
            id=json.getInt( name = "id"),
            title=json.getString( name = "title"),
            body=json.getString( name = "body")
        )

        Log.d(
            tag = "LAB11",
            msg = post.toString()
        )
    }

```

- And we should get something like:

```

con.example.apilab 0 {
    "userId": 1,
    "id": 1,
    "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
    "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas
}
con.example.apilab 0 sunt aut facere repellat provident occaecati excepturi optio reprehenderit
con.example.apilab 0 Post(id=1, title=sunt aut facere repellat provident occaecati excepturi optio reprehenderit, body=quia et suscipit
suscipit recusandae consequuntur expedita et cum
reprehenderit molestiae ut ut quas totam
nostrum rerum est autem sunt rem eveniet architecto)
con.example.apilab 0 Installing profile for con.example.apilab

```

-
- But as we need to define our function in a bit different way let's create the new Button so we need to "wrap up" our Buttons e.g. into the Column
- So we should have:

```

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView {
            Column() {
                Button(
                    onClick = {
                        Log.d("LAB11", msg = "Button clicked")
                        downloadPost()
                    }
                ) {
                    Text("Load data")
                }
                Button(
                    onClick = {
                    }
                ) {
                    Text("Load data to UI")
                }
            }
        }
    }
}

```

-
- Next we need a "UI-state-aware" variable e.g. as:

```

var text by remember { mutableStateOf( value = "No data") }

Button(
    onClick = {
    }
) {
    Text("Load data to UI")
}

```

-
- and we need the required imports to be added:

```

}

var text by remember { mutableStateOf( value = "No data") }

Button(
    onClick = {
    }
) {
    Text(
    )
}

```

Import operator 'MutableState.setValue'
Import operator 'State.getValue'
Create new scratch file from selection
Surround with widget
Adjust code style settings
Enable option 'Implicit receivers and parameters' for 'kotlin.jvm.functions.Function'

-
- Next we need a bit modified downloadPost implementation. To avoid any mistakes lets call it downloadPostForUI and it needs to be implemented in the following way:

```

fun downloadPostForUI(): String {
    try {
        val client = OkHttpClient()

        val request = Request.Builder()
            .url(
                "https://jsonplaceholder.typicode.com/posts/1"
            )
            .build()

        val response =
            client.newCall(request)
                .execute()

        return response.body?.string()
            ?: "No data"
    } catch (e: Exception) {
        return "Error: ${e.message}"
    }
}

```

-
- And now we may use it for our second Button in the following way:

```

var text by remember { mutableStateOf(value = "No data") }

Button(
    onClick = {
        Thread {
            val downloadedText = downloadPostForUI()

            runOnUiThread {
                text = downloadedText
            }
        }.start()
    }
) {
    Text("Load data to UI")
}

```

-
- And finally we may add the text element to our UI to display the downloaded content as:

```

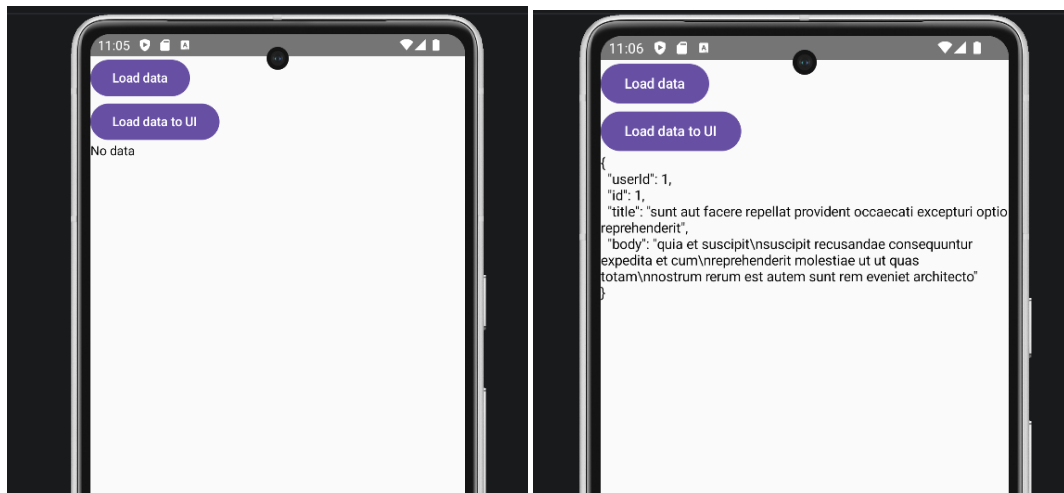
var text by remember { mutableStateOf( value = "No data") }

Button(
    onClick = {
        Thread {
            val downloadedText = downloadPostForUI()

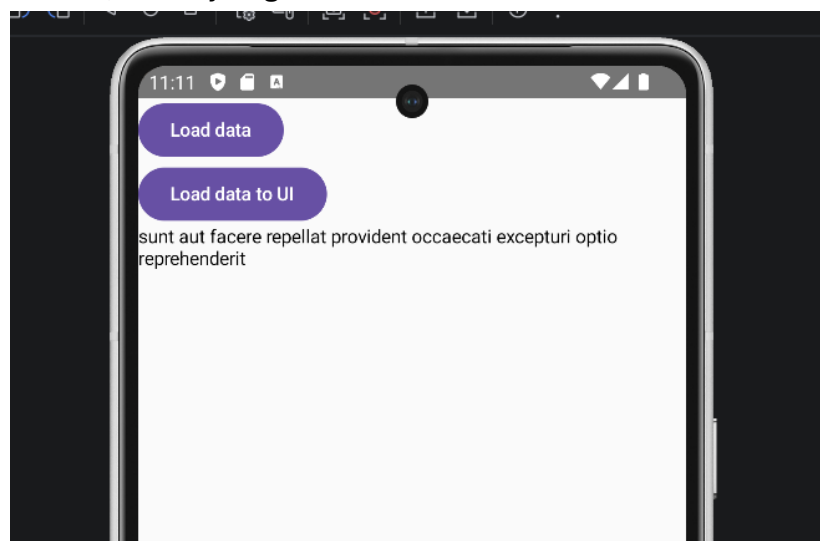
            runOnUiThread {
                text = downloadedText
            }
        }.start()
    }
) {
    Text("Load data to UI")
}
Text(text)
}

```

- With all those modifications we should get the following:



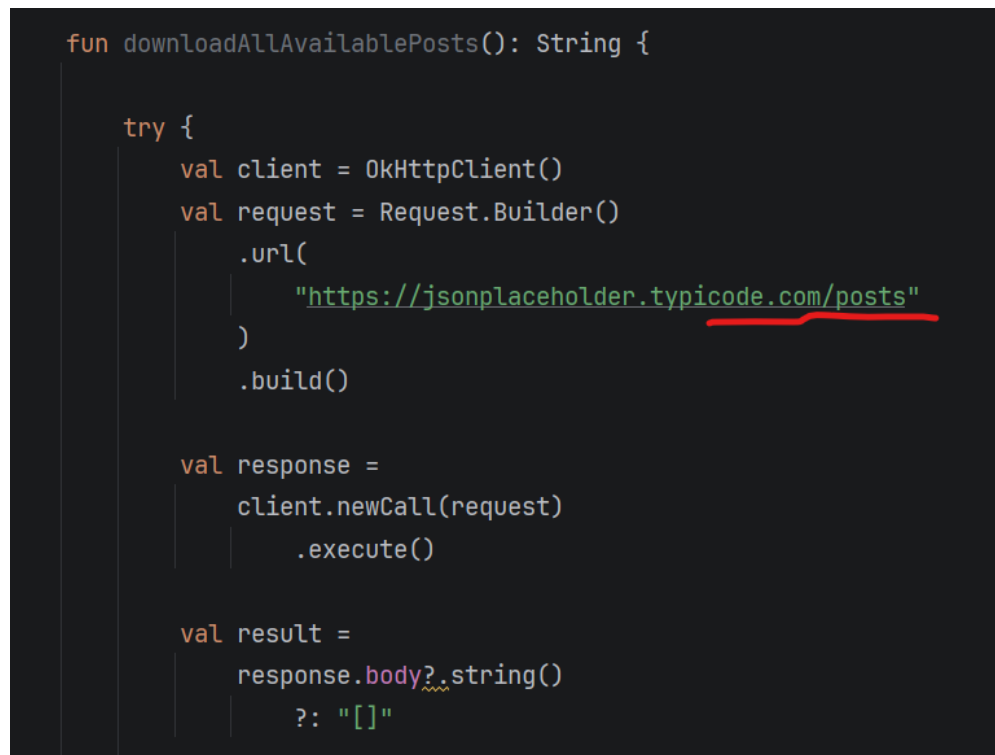
- Or even better, let's try to get:



- As the next step, let's try to get / work with the set / all the Posts returned by the API. Then again to not miss anything let's add a new Button:



- And let's build the new version of our “content-downloading” function and let's call it this time e.g. like downloadAllAvailablePosts
- The beginning will be almost the same as before except the endpoint address:



- Next as the response will be the set of posts not just the single Post we need the JSONArray and next let's create the Post objects based on the result we are getting:

```

val jsonArray =
    JSONArray(result)

val posts =
    mutableListOf<Post>()

for(i in 0 until jsonArray.length()) {

    val item =
        jsonArray.getJSONObject(index = i)

    val post = Post(
        id = item.getInt(name = "id"),
        title = item.getString(name = "title"),
        body = item.getString(name = "body")
    )

    posts.add(post)
}

```

- Let's maybe write the posts to the LogCat to see what we are getting:
-
- And as the result in the LogCat you should see something like:

The screenshot shows the LogCat interface in Android Studio. The log output displays a list of Post objects. Each object has an id, a title, and a body. The log output is as follows:

```

D (Post{id=1, title=sunt aut facere repellat provident occaecati excepturi optio reprehenderit, body=quia et suscipit
suscipit recusandae consequuntur expedita et cum
reprehenderit molestiae ut ut quas tota
nostrum rerum est autem sunt rem eveniet architecto), Post{id=2, title=qui est esse, body=est rerum tempore vitae
sequi sint nihil reprehenderit dolor beatae ea dolores neque
fugiat blanditiis voluptate porro vel nihil molestiae ut reiciendis
qui aperiam non debitis possimus qui neque nisi nulla), Post{id=3, title=ea molestias quasi exercitationem repellat
voluptatem occaecati omnis eligendi aut ad
voluptatem doloribus vel accusantium quis pariatur
molestiae porro eius odio et labore et velit aut), Post{id=4, title=eum et est occaecati, body=ullam et saepe reici
sit amet autem assumenda provident rerum culpa
quis hic commodi nesciunt rem tenetur doloremque ipsam iure
quis sunt voluptatem rerum illo velit), Post{id=5, title=nesciunt quas odio, body=repudiandae veniam quera
sunt ea

```

- Ok, then let's try to show the topics of all returned posts on the UI.
- To make it possible we need first the “state-aware” collection of titles:

The screenshot shows the Kotlin code for a state-aware collection of titles. The code is as follows:

```

override fun onCreate(savedInstanceState: Bundle?) {
    setContent {
        Column() {

            var titles by remember {
                mutableStateOf<List<String>>()
            }

            Button(
                onClick = {
                    Log.d(tag = "LAB11", msg = "Forth Button clicked")
                }
            ) {
                Text("Load many posts to UI")
            }
        }
    }
}

```

On the right side of the screenshot, there is a preview of a mobile app. The app has a white background with a list of buttons. The buttons are labeled: "Load data", "Load data to UI", "Load data", "Load many posts", and "Load many posts to UI".

- Next let's prepare the dedicated “content-dwonloading” function called e.g. downloadPostTitles where the beginning is almost the same as before but we need to return the collection of title so the full implementation may look as:


```

fun downloadPostTitles(): List<String> {
    return try {
        val client = OkHttpClient()
        val request = Request.Builder()
            .url("https://jsonplaceholder.typicode.com/posts")
            .build()
        val response = client.newCall(request).execute()
        val result = response.body?.string() ?: ""
        val jsonArray = JSONArray(result)

        val titles = mutableListOf<String>()

        for (i in 0 until jsonArray.length()) {
            val item = jsonArray.getJSONObject(index = i)
            val title = item.getString(name = "title")
            titles.add(title)
        }
        titles
    } catch (e: Exception) {
        listOf("Error: ${e.message}")
    }
}

```

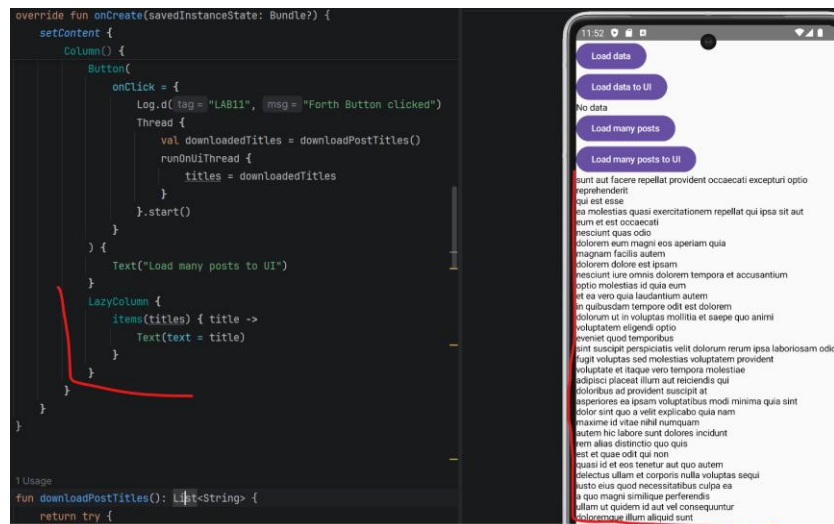
- Next, let's run this function correctly in our onClick handler:

```

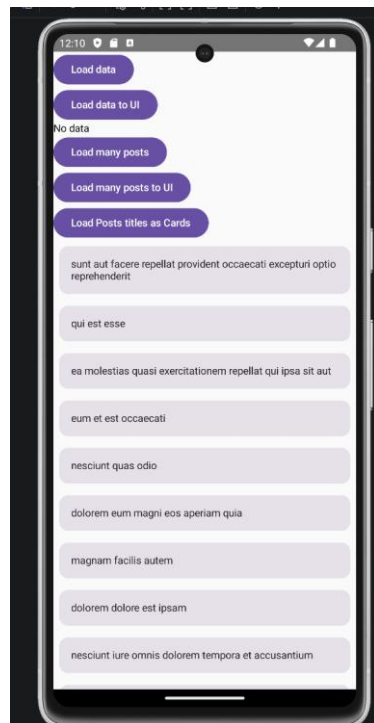
Button(
    onClick = {
        Log.d(tag = "LAB11", msg = "Forth Button clicked")
        Thread {
            val downloadedTitles = downloadPostTitles()
            runOnUiThread {
                titles = downloadedTitles
            }
        }.start()
    }
) {
    Text("Load many posts to UI")
}

```

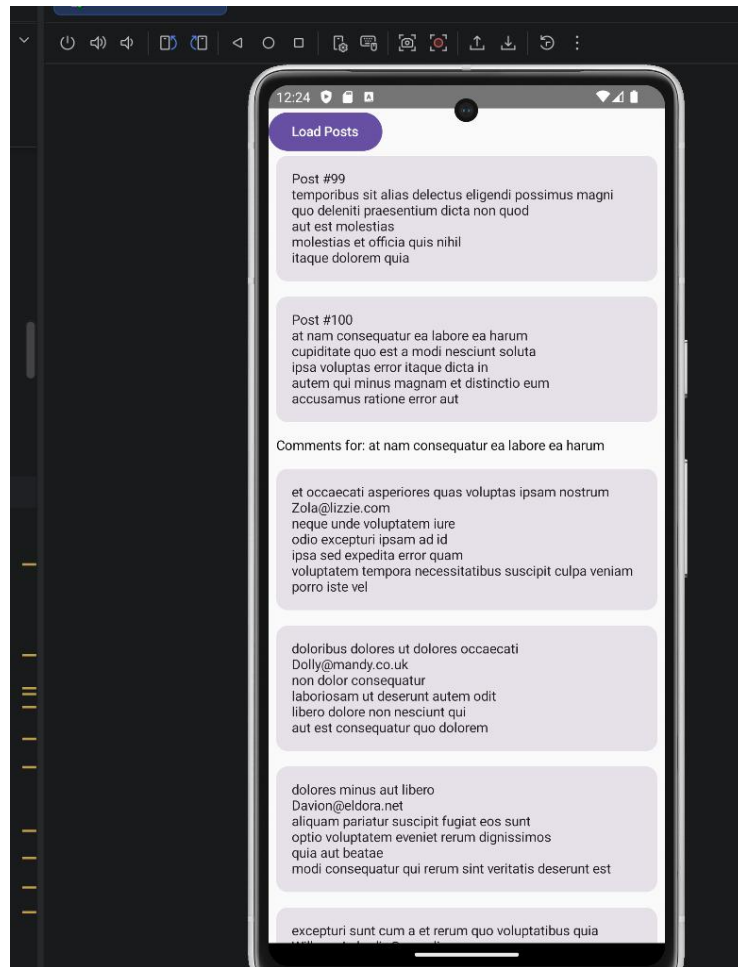
- And finally let's show the titles on the UI as:



- For practicing try to get something like:



- Optional task:
 - Try to get something like:



- I.e. the cards with the posts are “clickable” and after clicking the comments for the clicked posts are displayed
- When completed pls upload the screenshots / short videocast presenting how your app is implemented and how it works and upload to the upel platform as the solution for the current class.